Assignment-5
Theory

Tanu B
261100690012

command line arguments, fileoperators, read and filters

Write a Bash script that accepts a filename as the first command-line argument.
Check whether an argument is provided.
Check whether the file exists using a file operator.
If the file exists, display the first 5 lines using head.
Otherwise, display an appropriate error message.

#!/usr/bin/bash

Check if an argument is provided
if [$# -eq 0]; then
** echo "Error: No filename provided."**
** echo "Usage: $0 <filename>"**
** exit 1**
fi

FILE=$1

Check if the file exists
if [-e "$FILE"]; then
** echo "Displaying the first 5 lines of $FILE:"**
** head -n 5 "$FILE"**
else
** echo "Error: File '$FILE' does not exist."**
** exit 1**
fi

```
#!/bin/bash

if [ -z "$1" ]; then
    echo "Error: Please provide a filename."
    exit 1
fi

if [ -f "$1" ]; then
    head -n 5 "$1"
else
    echo "Error: File does not exist."
fi
```

File Type and Permission Checker

Write a Bash script that accepts a path as a command-line argument and determines whether it is:
a regular file,
a directory,
readable,
writable, and
executable.
Display suitable messages using file operators such as -f, -d, -r, -w, and -x.

```
#!/usr/bin/bash

if [ $# -eq 0 ]; then
   echo "Error: No path provided."
   echo "Usage: $0 <path>"
   exit 1
fi

TARGET=$1

if [ ! -e "$TARGET" ]; then
   echo "The path '$TARGET' does not exist."
   exit 1
fi

# Check file type
if [ -f "$TARGET" ]; then
   echo "'$TARGET' is a regular file."
elif [ -d "$TARGET" ]; then
   echo "'$TARGET' is a directory."
else
   echo "'$TARGET' is another type of file (e.g., symlink, device, etc.)."
fi

# Check permissions
if [ -r "$TARGET" ]; then
   echo "It is readable."
else
   echo "It is NOT readable."
fi

if [ -w "$TARGET" ]; then
   echo "It is writable."
else
   echo "It is NOT writable."
fi

if [ -x "$TARGET" ]; then
   echo "It is executable."
else
   echo "It is NOT executable."
```

fi

```
hdoop@Ubuntu:~/Desktop/Assignment5$ gedit file_check.sh
hdoop@Ubuntu:~/Desktop/Assignment5$ chmod +x file_check.sh
hdoop@Ubuntu:~/Desktop/Assignment5$ ls -l
total 12
-rwxrwxr-x 1 hdoop hdoop 359 Sep 13 00:02 cla1.sh
-rwxrwxr-x 1 hdoop hdoop 754 Sep 13 00:33 file_check.sh
-rw-rw-r-- 1 hdoop hdoop 412 Sep 12 23:46 pat.txt
hdoop@Ubuntu:~/Desktop/Assignment5$ mkdir check_permissions
hdoop@Ubuntu:~/Desktop/Assignment5$ ls
check_permissions  cla1.sh  file_check.sh  pat.txt
hdoop@Ubuntu:~/Desktop/Assignment5$ ./file_check.sh pat.txt
'pat.txt' is a regular file.
It is readable.
It is writable.
It is NOT executable.
hdoop@Ubuntu:~/Desktop/Assignment5$ ./file_check.sh check_permissions
'check_permissions' is a directory.
It is readable.
It is writable.
It is executable.
hdoop@Ubuntu:~/Desktop/Assignment5$ ./file_check.sh cla1.sh
'cla1.sh' is a regular file.
It is readable.
It is writable.
It is executable.
```

Write a Bash script that accepts a log file as a command-line argument and searches for the words: failed, denied, unauthorized, and error.

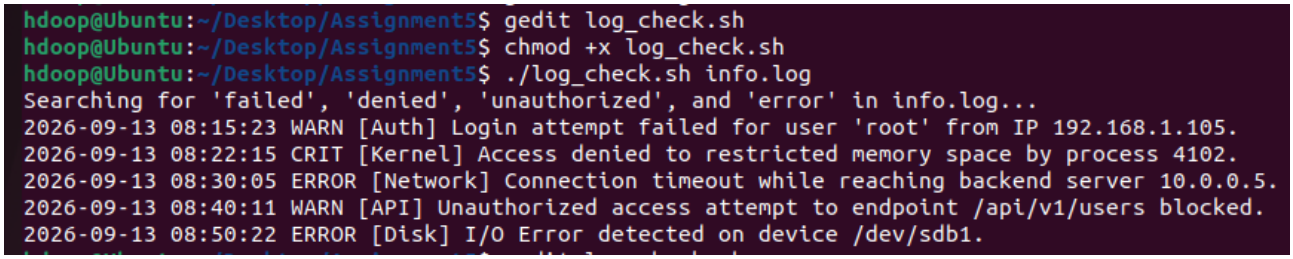
```
#!/usr/bin/bash

if [ $# -eq 0 ]; then
    echo "Error: No log file provided."
    echo "Usage: $0 <logfile>"
    exit 1
fi

LOGFILE=$1

if [ ! -f "$LOGFILE" ]; then
    echo "Error: File '$LOGFILE' not found."
    exit 1
fi

echo "Searching for 'failed', 'denied', 'unauthorized', and 'error' in $LOGFILE..."
# Using grep with -E for extended regex and -i for case insensitivity
grep -E -i "failed|denied|unauthorized|error" "$LOGFILE"
```



Write a Bash script that accepts a filename and checks whether:
the file exists,
it is a regular file,
it is empty or non-empty.
If it is non-empty, display the first 5 lines.

```
#!/usr/bin/bash

if [ $# -eq 0 ]; then
   echo "Error: No filename provided."
   exit 1
fi

FILE=$1

# Check if file exists
if [ ! -e "$FILE" ]; then
   echo "Error: The file '$FILE' does not exist."
   exit 1
fi
echo "The file exists."

# Check if it is a regular file
if [ ! -f "$FILE" ]; then
   echo "Error: '$FILE' is not a regular file."
   exit 1
fi
echo "It is a regular file."

# Check if it is empty or non-empty
if [ -s "$FILE" ]; then
   echo "The file is non-empty. Displaying the first 5 lines:"
   head -n 5 "$FILE"
else
   echo "The file is empty."
fi
```

```
hdoop@Ubuntu:~/Desktop/Assignment5$ gedit content_check.sh
hdoop@Ubuntu:~/Desktop/Assignment5$ chmod +x content_check.sh
hdoop@Ubuntu:~/Desktop/Assignment5$ ./content_check.sh pat.txt
The file exists.
It is a regular file.
The file is non-empty. Displaying the first 5 lines:
I am Gowtham
I Love Linux
I Like Crypto
I am studying at Manipal
I am a Cyber Security student
hdoop@Ubuntu:~/Desktop/Assignment5$ ./content_check.sh pattern.txt
Error: The file 'pattern.txt' does not exist.
hdoop@Ubuntu:~/Desktop/Assignment5$ gedit check1.txt
hdoop@Ubuntu:~/Desktop/Assignment5$ ./content_check.sh check1.txt
The file exists.
It is a regular file.
The file is empty.
hdoop@Ubuntu:~/Desktop/Assignment5$ ls
check1.txt  check_permissions  cla1.sh  content_check.sh  file_check.sh  info.log  log_check.sh  pat.txt
hdoop@Ubuntu:~/Desktop/Assignment5$ ./content_check.sh check_permissions
The file exists.
Error: 'check_permissions' is not a regular file.
```

Write a Bash script named investigate.sh that accepts a log file as a command-line argument.
The script must first verify that the file exists and is readable. Then ask the investigator to enter a search term.
Display the first 5 matching records and report the total number of matching records.

```
#!/usr/bin/bash

if [ $# -eq 0 ]; then
    echo "Error: No log file provided."
    echo "Usage: $0 <logfile>"
    exit 1
fi

LOGFILE=$1

# Verify the file exists and is readable
if [ ! -e "$LOGFILE" ]; then
    echo "Error: File '$LOGFILE' does not exist."
    exit 1
fi

if [ ! -r "$LOGFILE" ]; then
    echo "Error: File '$LOGFILE' is not readable by the current user."
    exit 1
fi

# Ask the investigator to enter a search term
read -p "Enter a search term: " SEARCH_TERM

echo ""
echo "--- First 5 Matching Records ---"
# Display the first 5 matching records
grep "$SEARCH_TERM" "$LOGFILE" | head -n 5
```

Report the total number of matching records
TOTAL_MATCHES=$(grep -c "$SEARCH_TERM" "$LOGFILE")
echo "Total number of matching records: $TOTAL_MATCHES"

